

Leva.Framework.Execution

`Leva.Framework.Execution` runs requested work on background tasks and tracks lifecycle, progress, cancellation, and typed results.

Execution provides reusable infrastructure for work that should not run inline inside application flow. It is intended for long-running calculations, import/export operations, external API calls, device operations, image processing, collision checking, and native or external delegated work.

Execution does not depend on Engine. Engine remains responsible for workflow orchestration through events, states, transitions, routines, behaviors, alarms, statuses, snapshots, and runtime history.

Execution is responsible only for running work and exposing its lifecycle and result.

Execution intentionally does not include external process execution, native interop helpers, process lifetime management, or workflow decisions. Those concerns belong to application code or separate framework libraries when they become necessary.

```
Leva.Framework.Execution  
-> Leva.Framework.Core
```

Project overview

The library separates requested work from the runtime attempt that performs it.

Application-specific request types describe what should be done. Execution types describe the attempt to run that request, track its current state, report progress, support cancellation, and return a typed result.

The central runtime objects are the execution board, execution runner, and execution handle. The board stores registered execution contracts and execution entries. The runner starts execution contracts on background tasks. The handle gives the caller access to one started execution.

Files and classes

Execution model

`ExecutionId` - Identifies one execution attempt. `ExecutionStatus` - Describes the lifecycle state of one execution attempt. `ExecutionEntry` - Stores the current snapshot of one execution attempt, including name, status, timestamps, latest progress, and optional error.

Execution contract

`IExecution<TRequest, TResult>` - Defines a typed execution contract. `ExecutionProgress` - Reports progress from inside a running execution.

Runtime coordination

`ExecutionBoard` - Stores registered execution contracts and current or completed execution entries. `ExecutionRunner` - Starts registered or directly supplied execution contracts on background tasks. `ExecutionHandle<TResult>` - Exposes one started execution to the caller.

Errors

`ExecutionErrors` - Creates common execution errors for missing registrations, cancellation, and failed executions.